

PSI3422 - Laboratório de Sistemas Eletrônicos

Relatorio final

Gabriel Ivo Bozi 14571845
Arthur Santos Alencar 13684745

29 de setembro de 2026

1 Aula 01

A nossa dupla se focou em separar cada parte do projeto em uma biblioteca e implementar elas na main, o código se encontra em anexo no repositório do github, neles criamos 2 bibliotecas uma para a leitura do sensor ultrassônico e outra para o controle da ponte H.

Obs: Os alunos optaram por não usar o platformIO e sim o zephyr de forma direta.

2 Aula 02

2.1 Objetivos

Implementar a comunicação serial entre duas placas de desenvolvimento **Freedom** (kl25z) utilizando os *transceivers* **nRF24L01**.

Através do terminal do computador, controlar remotamente um LED na outra placa **Freedom**.

2.2 Observações

- Lembre-se que a tensão máxima de alimentação do **nRF24L01** é de 3,6 V.
- Os transceptores utilizam a comunicação SPI.
- Além da comunicação SPI, os pinos **CE** e **CSN** (IRQ opcional) precisam ser configurados para transmitir/receber dados.

2.3 Resultados

A aula foi feita utilizando apenas os recursos do **Zephyr** (sem utilizar recursos *bare metal*), portanto para isso primeiramente foi feito um mapeamento dos pinos e recursos utilizados no **device tree** (**frdm_kl25z.overlay**).

Pino nRF24L01	Pino KL25Z	Configuração no Zephyr
SCK	PTC5	SPI0_SCK_PTC5
MOSI	PTC6	SPI0_MOSI_PTC6
MISO	PTC7	SPI0_MISO_PTC7
CSN	PTC3	GPIO_ACTIVE_LOW
CE	PTC9	GPIO_ACTIVE_HIGH
IRQ	PTD4	GPIO_ACTIVE_LOW GPIO_PULL_UP

Tabela 1: Mapeamento de pinos entre o módulo nRF24L01 e a placa Freedom KL25Z.

Em seguida foi feito um teste de *loopback* da comunicação SPI da placa para validar o funcionamento do recurso com isso se iniciou os estudos e desenvolvimento de uma biblioteca simples para controlar o módulo de rádio.

No rádio se utilizou os recursos de:

- ACK automático;
- 1 Mbps com 0 dBm na comunicação RF;
- Habilita o endereço de recepção no Pipe 0;
- Canal 0x10 de comunicação;

Com tudo configurado foi feita uma tentativa de comunicação entre a placa k125z e o nRF24L01, para validar a comunicação entre eles, para isso foi feita uma leitura do registrador 0x00 (CONFIG) esperando receber a resposta *default* 0x08.

Por fim, após todos os testes intermediários terem apresentados resultados positivos foi realizado a comunicação entre as placas com a instrução de ligar e desligar o LED verde da placa receptora, com sucesso.

3 Aula 03

A aula 3 foi focada na fabricação da PCI (placa de circuito impresso). principais problemas a freedon possui terminias de barramnetos paralelos isso dificulta muito a cooncepçao de uma PCI pois ao faszer o roteamneto muitas vezes passamos por cima de alguns terminais

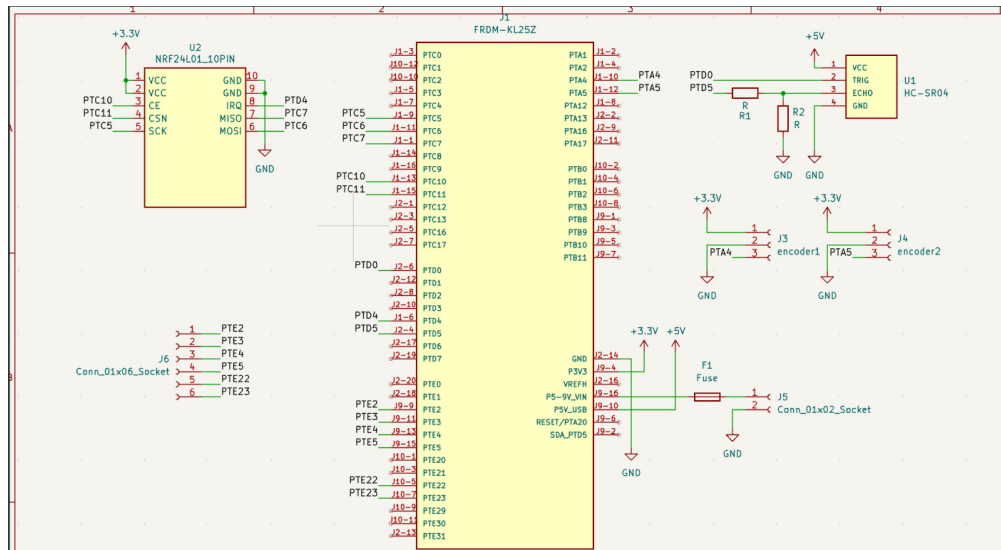


Figura 1: Esquemático no Kicad

4 Aula 04

Objetivo

Acoplar um encoder em cada motor, calibrá-lo para estimar a distância percorrida e usá-lo para controlar o movimento, realizando uma curva de 90° para a direita e outra para a esquerda.

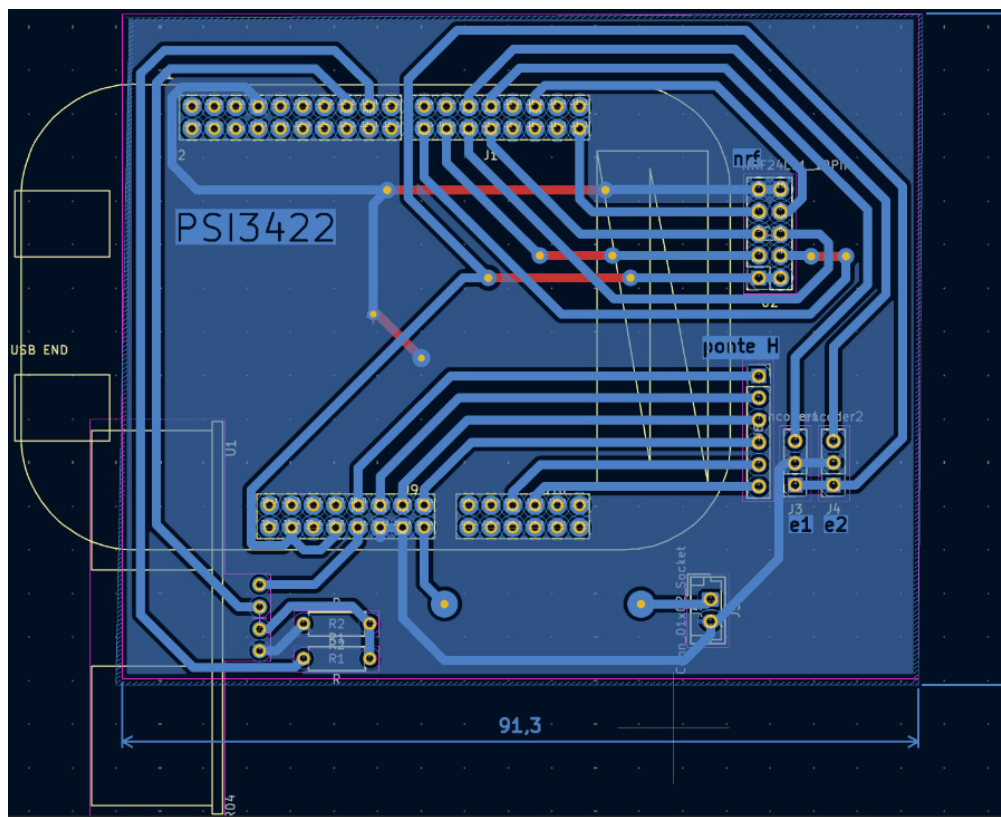


Figura 2: Footprint no Kicad



Figura 3: Placa finalizada com componentes

4.1 O que foi implementado, parcialmente implementado ou não implementado

Implementado.

- Leitura dos dois encoders (sensores IR HW-201 em PTA4 e PTA5) por interrupção de GPIO nas duas bordas, resultando em 48 pulsos por volta (24 faixas escuras e 24 claras), com um contador independente para cada roda.
- Filtro de *debounce* de 2 ms dentro da rotina de interrupção.
- Modo de alinhamento (`MODO_ALINHAMENTO`), que apenas imprime no terminal as transições de cada sensor, sem acionar os motores.
- Controle da ponte H L298N por quatro GPIOs (PTE2 a PTE5): frente, ré e parada de cada motor.
- Giro de 90° sobre o próprio eixo controlado pelos encoders: o alvo de pulsos é calculado por $\frac{\text{bitola} \cdot \text{pulsos por volta}}{4 \cdot \text{diâmetro da roda}}$ (bitola de 120 mm e roda de 66 mm, resultando em 21 pulsos por roda), e os motores param quando a média dos dois encoders atinge o alvo.
- Sequência de teste em *loop*: esquerda, direita, direita, esquerda, com intervalo de 10 s entre as ações e impressão dos pulsos contados a cada giro.
- Configuração do *overlay* de *device tree* (encoders e pinos da ponte H via *aliases*).

Parcialmente implementado.

- **Calibração do encoder:** a calibração é apenas teórica, baseada na geometria do carrinho (diâmetro da roda e bitola), sem escorregamento. O código deixa indicado que o valor deve ser ajustado manualmente após testes, e não há rotina de calibração experimental.
- Driver SPI próprio para o KL25Z e *binding* do nRF24L01+ estão presentes na pasta, mas não são compilados nesta aula (o `CMakeLists.txt` inclui apenas `main.c`). Servem de preparação para a etapa seguinte.

Não implementado.

- Estimativa da distância percorrida em linha reta: as constantes (diâmetro e pulsos por volta) existem, mas não há função que converta pulsos em distância nem deslocamento por distância medida.
- Controle de velocidade: os pinos ENA/ENB do L298N ficam fixos por jumper, então os motores operam apenas ligados ou desligados, sem PWM.
- Compensação de diferenças entre as duas rodas: o giro usa apenas a média dos dois encoders.

4.2 Dificuldades e soluções encontradas

- **Contagens falsas nos encoders** (ruído do comparador LM393 quando o sensor fica em distância ambígua).

Solução: *debounce* de 2 ms na interrupção, curto o bastante para não descartar pulsos reais do disco.

- **Alinhamento entre sensor e disco de faixas.**

Solução: modo de alinhamento que exibe em tempo real o estado de cada sensor, permitindo ajustar a posição mecânica antes de mover o carrinho.

- **Ângulo de giro dependente de escorregamento e de diferenças entre os motores.**

Solução parcial: modelo geométrico com uso da média dos dois encoders como critério de parada.

Dificuldades sem solução até a entrega. A precisão do ângulo de 90° e a estimativa de distância dependem de calibração experimental, que não foi concluída. Para resolver, seriam feitos: (i) medição do giro real em várias repetições e ajuste do número de pulsos alvo com base nos resultados; (ii) implementação da conversão $d = \frac{\text{pulsos}}{48} \cdot \pi D$ e validação medindo trajetos de comprimento conhecido; (iii) uso de PWM na ponte H para reduzir a velocidade durante os giros e diminuir o escorregamento.

5 Aula 05 e 06

Objetivo

Integrar encoders, ponte H, sensor ultrassônico e comunicação sem fio (nRF24L01+) em um carrinho que anda de forma autônoma, desvia de obstáculos e recebe comandos remotos.

5.1 O que foi implementado, parcialmente implementado ou não implementado

Implementado.

- Reorganização do código em bibliotecas separadas (`encoder`, `motor_control`, `ultrasound`, `nrf24l01`), com parâmetros configuráveis via Kconfig (papel de transmissor ou receptor, distância mínima de 10 cm, diâmetro da roda, bitola, pulsos por volta e tempo de parada).
- Controle de velocidade dos motores por PWM (canais do TPM2, período de 40 μ s, ou seja, 25 kHz), além dos comandos de frente, ré, esquerda, direita e parada.
- Sensor ultrassônico HC-SR04: pulso de *trigger* de 10 μ s gerado por PWM (período de 60 ms), medição do *echo* por interrupção com contador de ciclos, conversão para centímetros e notificação por semáforo quando a distância fica abaixo do limiar.
- Encoders com contadores atômicos e alvo de pulsos com notificação por semáforo, permitindo giros de ângulo arbitrário.
- Máquina de estados do carrinho (andar, desviar de obstáculo, parar), executada em *threads* com `k_poll`, semáforos e fila de mensagens. No desvio, o carrinho para, gira 90° para a esquerda e verifica o caminho; se ainda houver obstáculo, gira 180° para a direita e verifica novamente; caso persista, gira 90° para a direita e para.

Parcialmente implementado.

- **Enlace de rádio:** foram escritos o código de acesso por SPI ao nRF24L01+ (leitura e escrita de registradores, buffers e inicialização), o tratamento da interrupção IRQ, os papéis de transmissor e receptor escolhidos por Kconfig e o envio de comandos (R, S, P, C) digitados no terminal. No carrinho, apenas os comandos de *run* e *stop* têm efeito; P e C foram definidos, mas sem ação associada. Como a comunicação não funcionou, esta parte não pôde ser validada.
- **Calibração dos motores:** a função `motor_calibrate` apenas fixa as velocidades em 80% (esquerdo) e 100% (direito), sem rotina automática. As constantes de iterações e tolerância de pulsos estão declaradas, mas não são utilizadas.

Não implementado / não funcional.

- **A comunicação sem fio entre as placas não funcionou.** Consequentemente, o controle remoto do carrinho por rádio não pôde ser demonstrado.

5.2 Dificuldades e soluções encontradas

Principal dificuldade. O maior problema deste projeto foi o não funcionamento da comunicação sem fio entre as duas placas com os módulos nRF24L01+. A falha não foi solucionada até a data da entrega.

Hipótese para a causa e possível solução. Como possível causa, suspeita-se de mau contato nos *jumpers* usados para ligar os módulos de rádio à placa, já que o nRF24L01+ é sensível a conexões instáveis nos sinais de SPI e de alimentação. Essa suspeita é reforçada pelo fato de que, na Aula 2, a comunicação entre as placas funcionou corretamente, com o controle remoto do LED pelo terminal. Esta hipótese ainda não foi confirmada por testes.

Uma possível solução seria montar o módulo de rádio em uma PCB, eliminando os *jumpers* e deixando as conexões de SPI, CE, IRQ e alimentação fixas. Essa alternativa foi apenas considerada e não será implementada pelo grupo.

Outras dificuldades.

- **Sincronização entre interrupções e *threads*** (encoders, ultrassom e controle do carrinho).
Solução: contadores atômicos, semáforos de notificação, fila de mensagens e `k_poll` para o carrinho reagir a obstáculos e a comandos sem espera ocupada.
- **Medição de distância com o ultrassom** (conversão de ciclos do relógio para microssegundos e depois para centímetros, e ruído em pulsos muito curtos).
Solução: conversão com `k_cyc_to_us_floor32`, divisão por 58 e descarte de pulsos abaixo de uma duração mínima.
- **Motores com velocidades diferentes, fazendo o carrinho desviar da linha reta.**
Solução parcial: controle por PWM com velocidades distintas por lado (80% e 100%), ajustadas manualmente, sem calibração automática.

Anexos

Repositório do Github

<https://github.com/Arthur270903/PSI3422>